

1. La réversité, c'est quoi ?

1.1. Rappel : intérêt des fonctions.

Les fonctions permettent de décomposer un programme complexe en plusieurs sous-programmes plus simples. De plus, les fonctions sont réutilisables : si nous disposons d'une fonction capable de calculer une racine carrée, par exemple, nous pouvons la réutiliser à plusieurs endroits dans notre programme, sans avoir à la réécrire à chaque fois (on parle alors de **factorisation** du code)

- Que fait cette fonction ?

```
def f(x) :  
    return x+3
```

- Comment utiliser cette fonction dans un programme ?

- Les fonctions peuvent appeler d'autres fonctions : écrire fonction $g(x)$ qui permet de calculer $(x+3)^2$?

```
def f(x) :  
    return x+3  
def g(x) :  
    return f(x)*f(x)
```

Écrire une fonction $h(x)$ qui permet de calculer $2(6x-4)^3$, en utilisant une fonction $v(x)=6x-4$

<pre>def v(x) :</pre>	<pre>def h(x) :</pre>	<pre>def h(x) :</pre>
-----------------------	-----------------------	-----------------------

- Les fonctions peuvent s'appeler elles-même !

Une fonction peut également s'appeler elle-même. Les avantages de ce genre de construction peuvent, dans certains cas, permettre une écriture plus courte, si le pb est "récurif". Par exemple, considérons la fonction suivante :

1. Que va-t-il se passer pour `compte_a_rebours(3)` ?
2. comment faire pour éviter ce problème ? modifier le script en conséquence.

```
def compte_a_rebours(n) :  
    print(n,end="->")  
    compte_a_rebours(n-1)
```

```
def compte_a_rebours(n):
```

```
#version itérative
```

Une fonction qui s'appelle elle-même est dite **récurive** ; le processus de son exécution est appelé **récurivité**. Pour éviter que la récurivité se fasse à l'infini il faut atteindre le **cas de base** et préciser dans la définition de la fonction qu'**une fois le cas de base atteint le programme s'arrête**.

Un site pour visualiser les appels récurifs : <https://pythontutor.com/>

1.2. un exercice de récursivité sur les chaînes : le palindrome

Un palindrome est un mot ou une phrase pouvant être lue dans les deux sens comme "Hannah" ou "kayak". Écrire une définition récursive de la fonction palindrome, de paramètre une chaîne, qui renvoie True si la chaîne en est un, False sinon.



Lave mes péchés, pas seulement mon visage

2. Des fonctions récursives en maths

2.1. Définition du calcul de puissance, mais en plus rapide

On s'intéresse au calcul de x^n , n étant un entier positif, x un réel.

Proposer une fonction itérative, `puissance_iter(x,n)` qui renvoie x^n sans utiliser l'opérateur `**`.

Remarquons la chose suivante : $2^2 = 2*2$ et $2^3 = 2*2*2$ on peut aussi écrire $2^3 = 2*2^2$ ou encore $2^n = 2*2^{n-1}$ on peut alors écrire la définition récursive de la fonction puissance :

$$x^n = \begin{cases} 1 & \text{si } n=0 \\ x^n = x * x^{n-1} & \text{si } n>0 \end{cases} \leftarrow \text{le cas de base}$$

- Implémenter la fonction puissance avec un unique cas de base (le cas " $n=0$ "). x et n sont des paramètres.
- Implémenter la fonction puissance avec deux cas de base (" $n=0$ " et " $n=1$ "). x et n sont des paramètres.
- Implémenter la fonction `puissance_rapide` dont la définition récursive est la suivante :

$$x^n = \begin{cases} 1 & \text{si } n=0 \\ (x^2)^{\frac{n}{2}} & \text{si } n>0 \text{ et pair} \\ x * (x^2)^{\frac{(n-1)}{2}} & \text{si } n>0 \text{ et impair} \end{cases}$$

à l'aide des tests fournis, remarquer la différence de vitesse entre les fonctions

2.2. le nombre pi

François Viète a démontré en 1592 que l'on pouvait approcher le nombre π grâce à deux suites imbriquées (u_n) et (v_n) définies sur $\mathbb{N}$ par :

$$\begin{cases} u_0 = \sqrt{2} \\ u_{n+1} = \sqrt{2 + u_n} \end{cases} \text{ et } \begin{cases} v_0 = 2 \\ v_{n+1} = 2 \frac{v_n}{u_n} \end{cases} \text{ ou bien } \begin{cases} u_0 = \sqrt{2} \\ u_n = \sqrt{2 + u_{n-1}} \end{cases} \text{ et } \begin{cases} v_0 = 2 \\ v_n = 2 \frac{v_{n-1}}{u_{n-1}} \end{cases}$$

- Programmer la fonction récursive u , de paramètre n .
- Programmer la fonction récursive v , de paramètre n .
- Trouvez des valeurs approchées de π grâce à la fonction v .



2.3. Syracuse

Cette conjecture affirme que quelle que soit la valeur de u_0 , il existe dans la suite une valeur de n pour laquelle $u_n=1$. Cette conjecture défie toujours les mathématiciens, puisqu'il n'a pas encore été possible de prouver qu'elle est soit vraie, soit fausse, pour tout entier $\mathbb{N}$

$$u_{n+1} = \begin{cases} \frac{u_n}{2} & \text{si } u_n \text{ est pair} \\ 3u_n + 1 & \text{si } u_n \text{ est impair} \end{cases} \text{ avec } u_0 > 1 \text{ et } u_0 \in \mathbb{N}$$

écrire la fonction `syracuse(u_n)` qui affiche les valeurs successives de u_n , tant que $u_n > 1$



université de Syracuse

3. Exercices Turtle niveau 1:

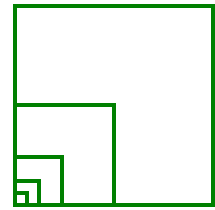
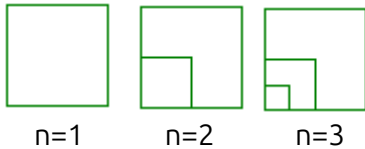
la doc complète quand vous voudrez mettre de la couleur :

<https://docs.python.org/fr/3/library/turtle.html>

3.1. carrés

Compléter la fonction `carre(cote)`, qui permet de tracer un carré de cote de longueur "cote"

On souhaite générer le motif ci-contre :



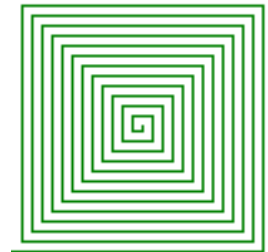
implémenter :

- la fonction `carre_rec(n,cote)` pour obtenir ce motif à l'aide d'une fonction récursive
- la fonction `carre_it(n,cote)` pour obtenir ce motif à l'aide d'une fonction itérative.

3.2. spirale

Programmer une fonction récursive spirale de déplacement réalisant la figure ci-contre. Le paramètre `n` de la fonction représente le nombre de segments de la spirale. Ici : `spirale(50)`

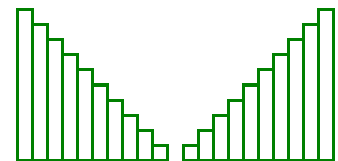
aide : mesurer la longueur des segments....



3.3. marches

Programmer une fonction `marche(l, h)`, qui trace un rectangle de dimension $l \times h$.

- Programmer deux fonctions récursives escaliers réalisant les figures ci-contre.



Si vous avez de l'imagination, vous pouvez imaginer de nombreuses autres figures !

4. Exercices Turtle niveau 2

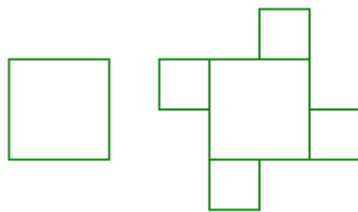
turtle_niveau2_eleve.py à lire et compléter

En 1975, Benoît Mandelbrot, mathématicien français exilé aux USA, publie un ouvrage intitulé « Les objets fractals » qui connaît un vif succès auprès de la communauté scientifique et aussi du grand public. Mandelbrot s'appuie sur des phénomènes naturels tirés de la nature (feuilles de fougères, aspect du chou romanesco, allure de la côte bretonne) pour désigner une nouvelle classe d'objets : la **fractale**, objet mathématique qui présente une structure similaire à toutes les échelles.



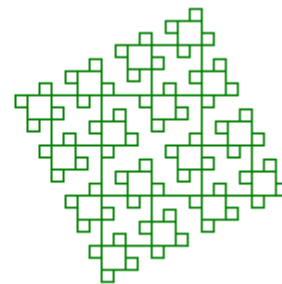
4.1. Les petits carrés

Voici le début d'une structure réursive : dessiner l'étape $n=3$



$n=1$

$n=2$



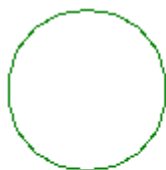
$n=3$

$n=4$

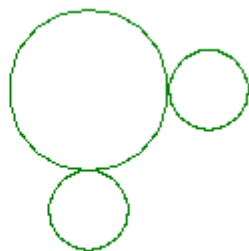
4.2. Des bulles

La figure de base est un cercle. Ensuite, on trace deux cercles réduits de moitié à droite puis en bas, tangents au cercle précédent. dessiner l'étape $n=3$

- Coder une fonction `cercle(x, y, r)` traçant un cercle de centre $(x; y)$ et de rayon r . la fonction `circle(r)` de turtle trace un cercle de rayon r (le centre se trouve à une distance r en dessous de la tortue)
- Déterminer, en fonction de x, y et r les coordonnées des centres des deux cercles réduits tangents.
- Programmer la fonction réursive `bulles(n, x, y, r)` réalisant la figure avec une profondeur n . Ci-dessous, les cas $n \in [1; 4]$.



$n=1$



$n=2$



$n=3$

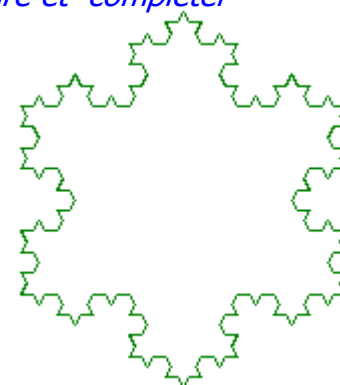
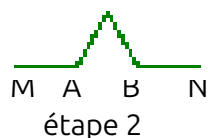
$n=4$

5. Exercices Turtle niveau 3 facultatif

[*turtle_niveau3_eleve.py à lire et compléter*](#)

5.1. le flocon de koch

En décomposant les étapes on obtient :



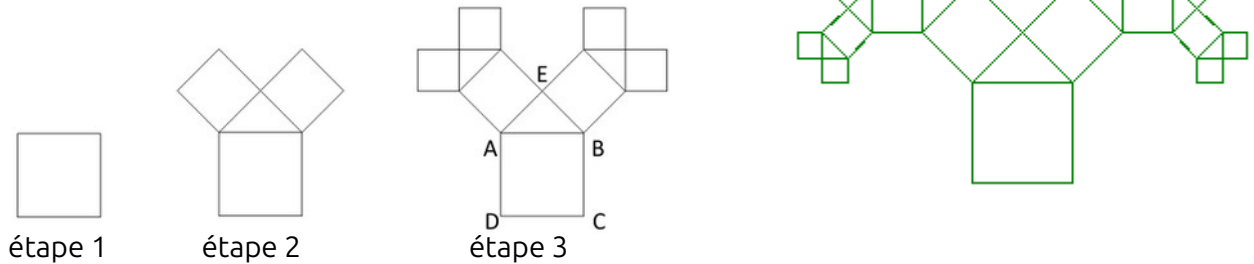
on ne dessine que des segments. Aide 1 : quelle est la longueur MA ?

Aide 2 : on applique la procédure koch sur chacun des 4 segments.

- Compléter la fonction `koch(n, long)`
- puis utiliser celle-ci pour dessiner un flocon

5.2. L'arbre de Pythagore

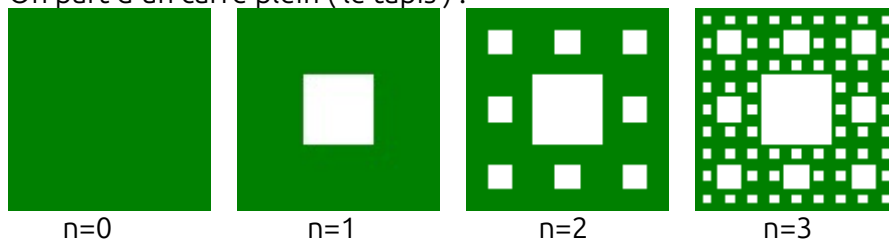
En décomposant les étapes :



on ne dessine que des carrés. Que vaut la longueur du coté du petit carré dans l'étape 2 ? écrire au brouillon les déplacements turtle à réaliser pour dessiner ces 2 carrés

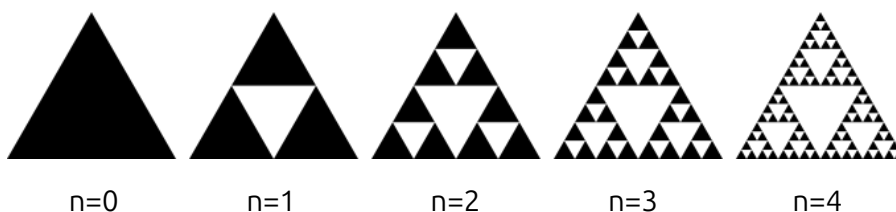
5.3. Le tapis de Sierpiński

On part d'un carré plein (le tapis) :



5.4. Le triangle de Sierpiński

On part d'un triangle plein équilatéral :



- on dessine un triangle blanc dans le triangle noir
- on remarque que de n=1 à n=2 la même figure est reproduite 3 fois
- il est plus facile de se déplacer avec des forward, backward, left et right

Remarque : en utilisant les couleurs rgb avec la fonction randrange, les figures seront plus jolies !